

APEX SNIFFER

Real-Time Network Packet Sniffer & Anomaly Detection System

ACADEMIC PROJECT REPORT

Course: B.Tech - Computer Science & Engineering

Author: Dashmeet Singh

Scope: Intrusion Detection System & Protocol Dissecting Core

Submission Date: June 2026

Abstract

Modern network infrastructures are susceptible to a wide variety of security threats, including unauthorized scanning, Denial of Service (DoS) attacks, and man-in-the-middle exploits. Traditional packet capture tools (such as Wireshark) offer robust protocol decoding but lack built-in real-time anomaly analysis and modern, lightweight application GUI frames suitable for cross-platform integration. This project presents **Apex Sniffer**, a native desktop Network Intrusion Detection System (NIDS) and protocol analyzer.

Apex Sniffer leverages a multi-threaded **Python background core** utilizing **Scapy** to capture packets on selected adapters and a **FastAPI** web server to perform live protocol analysis, traffic calculations, and signature-based threat detection. A modern frontend dashboard featuring **glassmorphism styling**, real-time charts (Chart.js), and Wireshark-like offset Hex views is served dynamically and wrapped inside a native GUI container using **PyWebView**. The system implements live anomaly checks for port scanning, TCP SYN floods, ARP spoofing, and DNS tunneling. Performance optimization is achieved by batching WebSocket broadcasts (50 packets every 100ms), protecting client-side rendering from CPU exhaustion under high traffic load.

1. Introduction & Objectives

In network security, passive monitoring is fundamental to detecting anomalies, troubleshooting connections, and analyzing protocol behavior. Packet sniffers intercept raw binary frames traversing network interfaces and reconstruct them into human-readable data layers.

1.1 Problem Statement

Existing network sniffers present several challenges:

1. **High Complexity:** Interfaces can be cluttered and hard to parse for junior security analysts.
2. **Lack of Automated Alerting:** Tools like Wireshark require manual writing of filters to catch attacks and do not highlight live threat actions dynamically.
3. **Dated UI Designs:** Traditional desktop interfaces (built on Tkinter, PyQt, or GTK) lack responsive, modern web-like styling, smooth animations, and interactive graph integrations.

1.2 Project Objectives

- Develop a **real-time packet capturer** that interfaces with physical and virtual NICs via raw sockets.
- Parse major protocol headers layer-by-layer: Ethernet, ARP, IP, IPv6, TCP, UDP, DNS, ICMP, and HTTP.
- Integrate a **Rule-Based Intrusion Detection Engine (IDS)** that logs threats dynamically.
- Implement a standalone **Desktop GUI Wrapper** running as a native Windows application.
- Build a responsive dashboard visualizing traffic rates (Throughput) and protocol distribution percentages.
- Design a fully custom Wireshark-equivalent **offset Hex & ASCII payload viewer** to allow deep packet investigation.

2. System Analysis & Requirements

2.1 Hardware Requirements

- **Processor:** Dual-Core 2.0 GHz or higher (x64 architecture).
- **RAM:** 4 GB minimum (8 GB recommended to buffer large packet history).
- **Network Interface Card:** Wi-Fi 802.11 or Ethernet adapter supporting promiscuous/monitor mode.

2.2 Software Requirements

- **Operating System:** Windows 10/11 (Administrator rights required for packet capture).
- **Environment:** Python 3.11+ (64-bit).
- **Libraries:** FastAPI (Web frame), Scapy (Sniffing engine), Psutil (Adapter retrieval), PyWebView (GUI wrapper).
- **Kernel Driver:** Npcap (Windows packet capture engine) installed in WinPcap-compatible mode.

3. System Architecture & Data Flow

The system employs a **decoupled Client-Server Architecture** operating as a single local process. The backend handles hardware-level packet interception, while the frontend handles rendering and user interaction.

- **Ingestion:** The user selects an adapter and clicks "Start". A background daemon thread initiates Scapy's socket sniffing loop.
- **Analysis:** For each frame captured, the raw bytes are parsed by the ``PacketAnalyzer``. It increments statistics, logs bandwidth data points, and matches packet attributes against threat thresholds.
- **Buffering:** The processed packet dictionary is added to a thread-safe ``Queue`` (maximum capacity 2,000 to prevent memory exhaustion).
- **WebSocket Streaming:** An asynchronous task drains the queue. To avoid browser lag, packets are compiled and transmitted in batches of up to 50 packets every 100ms. Cumulative statistics and threats are broadcast once per second.
- **Rendering:** The Javascript frontend appends packet rows to the scrollable table, pushes byte counts to Chart.js datasets, and checks search query filters in real-time.

4. Component Implementation Details

4.1 Multi-Threaded Sniffer Core

Sniffing network traffic is a blocking CPU-bound loop. To prevent it from freezing the web server API, the sniffer operates on a dedicated daemon thread. It uses Scapy's ``sniff()`` engine, which interfaces directly with Npcap.

- **Interface Resolution:** On Windows, adapters are loaded in the registry format (GUIDs, e.g. ``{GUID}``). The sniffer automatically maps this GUID to the kernel device path (``\\Device\\NPF_{GUID}``) in ``conf.ifaces``, avoiding Win32 directory path syntax errors (Code 123).
- **PCAP Export:** Scapy's ``wrpcap`` saves the memory packet buffer into a standard ``pcap`` file, allowing users to export capture sessions directly into Wireshark.

4.2 Traffic & Threat Analyzer

This module decomposes packet layers and evaluates security signatures:

- **IPv4/IPv6 Decoder:** Extracts source/destination IPs, TTL, fragmentation parameters, and checksums.
- **Transport Decoders:** Parses TCP flags (SYN, ACK, FIN, RST, PSH, URG), sequence numbers, source/destination ports, and UDP length.
- **Application Decoders:** Resolves DNS query names and decodes raw HTTP request headers (GET, POST requests).
- **Bandwidth Tracker:** Maintains a 3-second sliding window of byte sizes to calculate accurate live packets/sec and throughput bytes/sec rates.

4.3 Web Server & Desktop Wrapper

- **FastAPI Server:** Coordinates control endpoints (``/api/start``, ``/api/stop``, ``/api/clear``, ``/api/download-pcap``) and handles WebSocket handshakes (``/ws``).
- **Desktop Frame:** ``pywebview`` initiates a native OS window container. By pointing to ``http://127.0.0.1:8000/``, it serves the HTML dashboard as a standalone desktop utility, locking out default browser chrome elements (URL bars, bookmarks, developer overlays).

5. Network Intrusion Detection (IDS) Algorithms

The NIDS engine implements five rule-based detection models to identify suspicious network activity.

5.1 Port Scanning Detection

An attacker scan determines which service ports are active on a victim machine by querying multiple ports in rapid succession.

- **Algorithm:**
 - Track incoming TCP/UDP connections grouped by source IP address.
 - Store destination ports in a unique set.
 - Reset the set if the interval between connections from that source IP exceeds 10 seconds.
- **Alert Signature:** If ``len(ports_set) > 15`` within 10 seconds, raise a `***Port Scanning Detected***` alert.

5.2 TCP SYN Flood Anomaly

A Denial-of-Service technique where an attacker floods the target with TCP SYN requests without completing the three-way handshake, exhausting server connection slots.

- **Algorithm:**
 - Check for packets with TCP flags containing ``"S"`` (SYN) and omitting ``"A"`` (ACK).
 - Maintain a count of SYN packets from each source IP.
 - Reset the count every 5 seconds.
- **Alert Signature:** If ``syn_count > 50`` within 5 seconds, raise a `***SYN Flood Anomaly***` alert.

5.3 ARP Spoofing (Man-in-the-Middle)

An attacker associates their MAC address with the IP address of a legitimate gateway or host by sending fake ARP replies.

- **Algorithm:**
 - Monitor ARP reply packets (``op == 2``).
 - Maintain a local key-value table linking ``IP_Address -> MAC_Address``.
 - For each reply, compare the sender's MAC with the cached MAC for that sender's IP.
- **Alert Signature:** If ``cached_mac != sender_mac`` for a given IP, raise a critical `***ARP Spoofing Detected***` alert.

5.5 DNS Tunneling Anomaly

DNS tunneling encodes malicious payload data inside DNS domain queries to bypass firewalls and establish Command and Control (C2) communications.

- **Algorithm:**
 - Intercept DNS queries.
 - Measure the character length of the queried domain name.
- **Alert Signature:** If ``len(query_domain) > 80``, raise a `***Suspicious DNS Tunneling***` alert.
- Check domain endings. If domain matches malware-associated TLDs (e.g. ``top``, ``xyz``, ``download``), log a warning.

6. Testing, Performance & Results

6.1 Functional Verification

- **Real-time Streaming:** Pinging local hosts (ICMP) and accessing websites (HTTP/HTTPS) immediately populated the live table.
- **Hex / ASCII Inspection:** Clicking on a packet displayed a structured tree decode. The hex dump correctly formatted 16 bytes per line with standard offset counters.
- **Intrusion Alerts:** Simulated port scanning (using custom scripts) triggered red high-priority alerts within 2 seconds of launching the scan.

6.2 Performance Tuning

In network sniffing, packet ingestion rates can reach up to 10,000 packets/sec on busy networks. Initial builds that streamed packets individually over the WebSocket caused the frontend browser to freeze due to DOM rendering overhead.

- **Batching Solution:** Packets are buffered and pushed over WebSockets in batches of up to 50 every 100ms.
- **DOM Recycler:** Capped the scrolling table at 1,000 rows. Older rows are removed as new ones arrive, preventing browser crashes.

7. Conclusion & Future Scope

7.1 Conclusion

Apex Sniffer successfully implements a real-time network analyzer wrapped in a native desktop framework. It demonstrates how multi-threaded Python cores can be paired with high-performance web frontends to build security utilities that out-perform traditional GUI libraries (like Tkinter) in terms of visual presentation, responsiveness, and statistical charting.

7.2 Future Scope

- **Deep Packet Inspection (DPI) of Encrypted Data:** Adding decryption hooks for TLS traffic using pre-master secret key logs.
- **Machine Learning NIDS:** Replacing static signature thresholds with clustering models (e.g. Isolation Forest, K-Means) trained to spot network anomalies dynamically.
- **Automated Mitigation:** Implementing active defensive rules, such as triggering Windows Firewall commands to block offending IPs when critical threats (like SYN floods) are detected.